

Replayable Trajectory Dataflow: Auditing Reward Measurement under Optimization and Drift in RLVR

Zepeng Zhang

The University of Osaka

zhang.zepeng@ist.osaka-u.ac.jp

 <https://github.com/PeterWrighten/rtd>

Abstract

Reinforcement learning with verifiable rewards (RLVR) optimizes policies against measurements that can be exploited or silently changed. After a failure, checkpoints preserve training state but do not identify which historical measurements influenced it. We introduce Replayable Trajectory Dataflow (RTD), a retention and lineage contract for retrospective audits. RTD preserves evidence required by a declared query class, links versioned signals to consuming updates, and exposes rescore, diff, and localize operations under a pinned reference. We formalize its dependency closure and distinguish a reference-unaffected rollback boundary from a performance-optimal recovery checkpoint. Controlled arithmetic experiments expose a degraded 13.7% subpopulation hidden by aggregate accuracy. A 41,600-trajectory VeRL/Qwen2.5-Math-1.5B incident requires 2.99 seconds of CPU query stages without new sampling. After 40 fresh recovery updates, the boundary and latest-checkpoint arms reach 62.5% and 0% final-batch reference accuracy; a later, partially independently audited arm reaches 76.25% from an affected intermediate checkpoint. Three paired single-GPU trials detect no raw-capture slowdown. A warm-filesystem query benchmark using precomputed reference labels processes one million synthetically replicated trajectories in 44.21 seconds. These results support RTD as an audit substrate while separating dependency exclusion from recovery utility: its guarantees depend on capture completeness and the chosen reference, and do not establish optimal repair or detect failures shared by that reference.

1 Introduction

In reinforcement learning with verifiable rewards (RLVR) (DeepSeek-AI, 2025), the evaluator is part of the learning loop. A policy produces an answer, a verifier assigns reward, and optimization changes the distribution of future answers. Reward therefore serves two roles: a measurement of task success and the signal against which the policy is optimized. These roles can diverge. An exploitable proxy rewards behavior that does not solve the task; a verifier deployment changes the effective objective; or rollout and training engines disagree about the probability of the same actions. Evidence of reward hacking in production RL and of monitoring evasion makes this more than a hypothetical concern (MacDiarmid et al., 2025; Baker et al., 2025).

Once an operator notices a failure, a second problem begins: determining what happened before the alarm. A checkpoint tells the operator where execution can restart, but does not by itself identify which scores were wrong or which updates used them. Aggregate accuracy can remain healthy when only a small input slice degrades. Evaluating a checkpoint on newly sampled answers measures its present behavior, not the historical outputs whose rewards shaped it. Such evaluations can be useful for selecting a recovery policy, but they answer a different question from an audit of consumed measurements.

We propose *Replayable Trajectory Dataflow* (RTD), a lineage abstraction that makes those historical questions explicit. Its organizing rule is to partition state by *recomputability relative to a query class*. Retain sampled outputs, consumed measurements, and execution observations that the intended queries cannot reconstruct; recompute other quantities only when their dependencies are available and pinned. This is not a claim that randomness makes replay impossible. Exact replay can work when random state and execution are sufficiently controlled. Rather, a checkpoint and a sampling configuration do not themselves supply such a guarantee, and a retrospective audit should not silently substitute new outputs for missing historical evidence.

The resulting interface has three operations. RESCORE applies a replacement instrument to retained trajectories. DIFF compares its readings with the signals recorded during training. LOCALIZE follows consumption and training-state dependencies from the disagreements to updates and saved checkpoints. A query result includes both the chosen reference and the disagreement rule. It can be revised when a better reference becomes available without changing the underlying evidence.

The central distinction is between *dependency status* and *behavioral quality*. A checkpoint outside the resulting closure has no recorded path from a disputed measurement, subject to capture completeness. A checkpoint inside the closure may nevertheless perform well or recover quickly. Closure is useful when the recovery requirement is to exclude dependence on disputed signals; it is not a ranking of future task accuracy. This distinction matters empirically: in the retained VeRL incident, a later recovery from an intermediate affected checkpoint reaches higher final-batch accuracy than the arm restored from the last unaffected checkpoint. The supplementary status of that later run limits the comparison, but the outcome illustrates why affected ancestry alone does not establish poor recoverability.

This paper contributes (1) a query-relative retention contract, including explicit signal identity and completeness assumptions; (2) a formal reference-relative closure and rollback rule, with limits on what they prove; (3) a mapping from that contract to an offline JSONL sidecar and its narrower sequential implementation; and (4) an evaluation separating mechanism checks, recovery outcomes, physical storage, capture cost, and query scaling. The evidence combines controlled GRU experiments with one injected verifier-regression incident on a 1.5B-parameter model. These experiments establish feasibility and expose important limits, not the prevalence of failures or efficacy at frontier scale.

The practical value is retrospective access to the evidence actually used by optimization. In the subgroup experiment, the recorded mismatch set lies entirely in an affected input slice even though overall corrected accuracy is stable. In the VeRL case, a 91.49 MB sidecar supports reference evaluation and lineage queries in 2.99 seconds of measured CPU query stages. Obtaining a valid reference, capturing complete evidence, and deciding how aggressively to recover remain separate responsibilities.

2 Problem Model and Audit Questions

2.1 Measurements inside optimization

Let x be a prompt, y a sampled response, and π_θ the behavior policy. A verifier version v produces a signal $r_v(x, y)$, which a training algorithm consumes to update policy and optimizer state. In GRPO, rewards within a prompt group determine relative advantages (Shao et al., 2024). Changing one reward can therefore affect gradients associated with other members of the group. Recording only the individually disputed trajectory is not enough to establish an exact per-example parameter effect; the consuming update is a natural unit for conservative dependency analysis.

We distinguish three audit questions. *Measurement disagreement* asks whether a pinned replacement instrument gives a different reading on the same retained event. *Dependency localization* asks which updates and checkpoints have recorded paths from those disagreements. *Recovery utility* asks which action will best restore performance under a resource

budget. RTD directly specifies the first two and provides constraints for the third. It does not learn an optimal recovery policy.

2.2 Failure families

Optimization against a proxy. A policy can learn to satisfy an answer-format or substring condition without solving the task. The verifier need not change: optimization changes which parts of its decision surface are visited. Historical rescoring can reveal divergence if the replacement reference distinguishes the exploited behavior. It cannot reveal a failure that both instruments accept. In particular, “corrected” denotes an operator-supplied reference, not externally proven access to the intended construct.

Instrument drift. The evaluator changes during a run. Code, dependencies, answer extraction, or reward aggregation may be responsible. Version metadata can identify when a deployment occurred, but a version boundary alone does not prove that a consumed score changed. Conversely, a score disagreement can occur without a recorded version bump. RTD keeps both version identity and readings, enabling operators to compare versions on fixed events and to locate the first *observed consumed disagreement*, which need not be the time of the underlying deployment.

Engine-trainer mismatch. The rollout engine and trainer can assign different log-probabilities to the same sequence. Routing replay and mismatch diagnosis have been studied specifically for LLM RL (Ma et al., 2025; Zhong et al., 2026). The RTD interface can compare a retained engine-side signal with a trainer-side recomputation. The trainer reading is not inherently the ground truth; the query must state which discrepancy is meaningful. Token-level comparisons require exact tokens, alignment, and conditioning state, while replaying expert routing additionally requires the relevant routing decisions. Our real LLM incident evaluates verifier queries, not this broader engine contract.

2.3 What is held fixed in a counterfactual query?

Historical rescoring computes $r_{v^*}(x, y)$ with x and y fixed. It does not sample a hypothetical response from a policy that would have been trained under v^* all along. It therefore answers a measurement counterfactual, not the causal counterfactual “what would the final model have been without the fault?” Obtaining that model requires training or additional assumptions that RTD does not supply. Similarly, a difference in a scalar reward does not establish a nonzero gradient difference: clipping, group normalization, or zero learning rates can erase its numerical effect. Dependency closure deliberately retains these potentially affected descendants.

2.4 Operational assumptions

An audit selects a finite retained history and a fixed query recipe: reference code and dependencies, signal type, disagreement predicate, and capture scope. The supported reference must be evaluable from the retained payload. A tool-using trajectory may require external responses and environment observations in addition to model text. If these inputs were not retained, the appropriate result is an incomplete audit, rather than treating a newly queried environment as historical evidence.

We assume the recorder is trusted to report consumption faithfully. An append-only convention is not a tamper-evident log, and the prototype does not protect against malicious deletion or fabricated provenance. The formal results below require complete captured dependencies, including policy and optimizer-state continuation. The implementation evaluation uses a single sequential training history; arbitrary asynchronous branching is covered by the specification only when those actual edges are available.

Evidence	Required purpose	Retention boundary
Prompt, response, task context	Re-evaluate the identical historical event	Store text for text grading; retain exact tokens for token queries
Consumed signal and producer identity	Compare the actual training input with the reference	Retain value, type, version, and signal identifier
Consumption and state edges	Trace disputed inputs to updates and saved state	Retain actual consumers, not only generation timestamps
Behavior log-probabilities / routing	Compare engine readings or replay routing	Retain when not reconstructable and required by the query
Trainer quantities	Recompute likelihoods or advantages	Derive only with all conditioning, grouping, state, and execution dependencies
Reference outputs and closures	Repeat a pinned audit efficiently	Derived cache; never overwrite the original signal

Table 1: A query-specific retention contract. Optional engine fields are not part of the measured VeRL text-grading sidecar.

3 Retention and Lineage Contract

3.1 Evidence sufficient for a query class

Let h denote an execution history, $B(h)$ its retained base evidence, and D the pinned external dependencies (code, datasets, tokenizer, and recoverable states). A query class $\mathcal{Q}$ is supported when every $q \in \mathcal{Q}$ can be evaluated from $(B(h), D)$ without substituting unobserved events. This definition allows different physical encodings: text may suffice for a text verifier, whereas exact token likelihoods require the actual token sequence and its conditioning context.

Proposition 1 (Indistinguishable histories). *If two histories h_1, h_2 have the same retained evidence and pinned dependencies but $q(h_1) \neq q(h_2)$ for some $q \in \mathcal{Q}$, no algorithm using only that retained evidence can correctly answer q on both histories.*

Proof. The algorithm receives identical inputs for both histories, so it cannot return two different guaranteed-correct answers. A randomized algorithm has the same output distribution and likewise cannot guarantee both. $\square$

This elementary condition explains the materialization boundary. If sampled outputs or consumed scores distinguish the histories relevant to a query, some sufficient representation of them must survive. It does not prove that every field in our schema is byte-minimal. Deduplication, compression, and deterministic reconstruction can all reduce retained bytes without changing distinguishability. Nor does the condition force logging entire execution states: the contract is restricted to the questions being supported.

3.2 Typed execution graph

The full specification uses four node sets: trajectories T , signals S , updates U , and training-state versions C . A state version includes policy parameters and the optimizer or auxiliary state needed for continuation. Only $C_{\text{saved}} \subseteq C$ has a recoverable artifact. Every node belongs to a run and has an immutable identifier. A signal $s \in S$ records its subject $t(s)$, value $r(s)$, signal type, and producer version. Multiple signals may refer to one trajectory.

The directed acyclic graph $G = (T \cup S \cup U \cup C, E)$ contains the dependencies actually used: a behavior state to a sampled trajectory, a trajectory to its signal, a consumed trajectory or signal to an update, the update’s input state to that update, and an update to its output state. Edges may also represent auxiliary inputs such as reference-model or critic state when these are part of the supported analysis. A logical state need not be saved on disk to carry a dependency. Figure 1 gives the overall RTD workflow. Its store illustrates the general

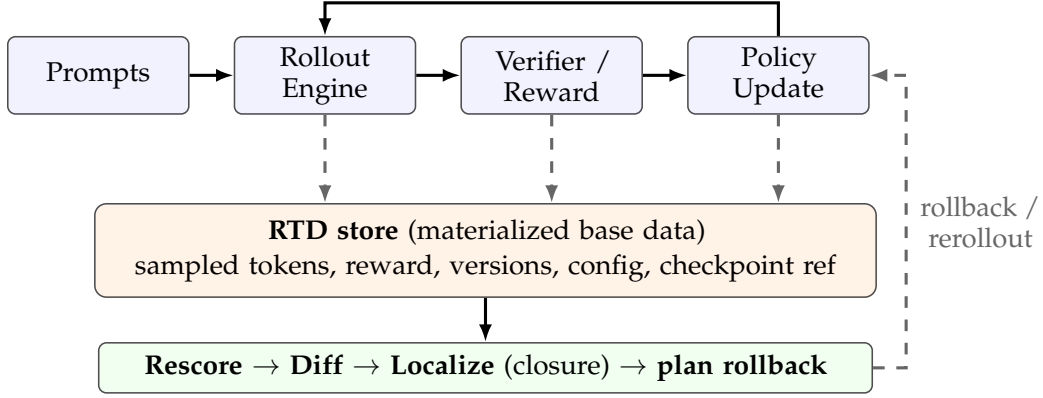


Figure 1: RTD materializes evidence (dashed) from the rollout, verifier, and update stages; rescore/diff/localize query the store to diagnose a measurement failure and select a rollback checkpoint that the loop re-runs from.

retention contract; the evaluated text-verifier profile retains decoded text, as detailed in Section 4.

3.3 Reference-relative mismatch and closure

A recipe $\rho = (f^*, P, S)$ pins a reference evaluator, a Boolean disagreement predicate, and an audited set of recorded signals. Define

$$r_\rho^*(s) = f^*(t(s), \text{context}(s)), \quad (1)$$

$$M_\rho = \{s \in \mathcal{S} : P(r(s), r_\rho^*(s)) = 1\}, \quad (2)$$

$$K_\rho = \text{Reach}_G(M_\rho) \cap (U \cup C). \quad (3)$$

Here reachability follows outgoing dependency edges. For scalar rewards, P may compare absolute differences against a tolerance, or compare thresholded pass/fail decisions. These predicates are not interchangeable. The VeRL incident uses pass/fail disagreement at 0.5; it does not enumerate every changed continuous reward.

Signal identity avoids an important ambiguity. A trajectory can be scored by several verifier versions, but only one reading may enter a particular update. A mismatch on an unused score must not seed that update. The contract therefore seeds *signals*, not an unqualified trajectory identifier. When each trajectory has one audited signal and every consuming update uses that signal, the trajectory projection used by our sequential prototype yields the same affected update/checkpoint set.

Proposition 2 (Reference-relative localization). *Assume every audited consumed signal and all relevant downstream consumption and state dependencies are captured faithfully, reference evaluation succeeds on the retained inputs, and ρ is fixed. Traversal from M_ρ returns exactly the recorded updates and states with a dependency path from a disagreement under ρ .*

Proof. A traversal marks a seed and then only its successors, so every marked node has a path from a seed. Conversely, induction on path length shows that every node reachable from a seed is eventually marked. Intersecting with $U \cup C$ gives the stated set. Capture completeness transfers this recorded-graph statement to the specified dependency model of the run. $\square$

The result is intentionally structural. It does not prove the reference is correct, that an affected update changed the numerical state, or that a later checkpoint has lower accuracy. A missing dependency can produce false negatives. Spurious edges can enlarge the closure. If the reference shares the original instrument’s blind spot, there may be no seed at all. A query should return its coverage conditions alongside the result.

3.4 Localization and rollback algorithm

For a sequential branch, let u_j be the earliest update in K_ρ . The policy *exclude disputed dependencies* selects

$$c^* = \arg \max_{\substack{c \in C_{\text{saved}} \cap \text{Anc}(u_j) \\ c \notin K_\rho}} \text{order}(c).$$

No reachable update means no rollback is prescribed. If a disagreement was consumed but no qualifying saved state exists, recovery requires an initial state or an external restoration procedure. For multiple branches, selection is performed within the branch being restored.

Audit and dependency-excluding rollback algorithm

1. Validate the retained scope, signal identities, and dependency coverage. Pin the reference, predicate, and selected branch.
2. RESCORE each required trajectory/context and retain the result as derived state under the recipe identity.
3. DIFF each audited recorded signal against that result to form M_ρ ; report evaluation failures separately from matches.
4. LOCALIZE by a worklist traversal of outgoing dependencies from M_ρ , deduplicating visited nodes, to obtain K_ρ .
5. If an update is affected, return the latest eligible saved ancestor outside the closure, or “restart required.” Otherwise return “no rollback under this recipe.” Return scope and assumptions in all cases.

Proposition 3 (Maximal retained prefix). *On a complete sequential history, if u_j is the first affected update, all subsequent states depend on u_j through state continuation. The latest saved ancestor outside K_ρ retains the longest saved prefix satisfying exclusion of the recorded disputed dependencies.*

Proof. Each later update consumes the preceding state, yielding a path from u_j to every later state. Any saved state after u_j is therefore ineligible. Among eligible saved ancestors, selecting the latest retains the longest prefix by the total ordering of the branch. $\square$

This is optimal only for the stated prefix constraint. It makes no claim about expected recovered accuracy, wall-clock repair time, or the best use of a limited training budget. A different policy can deliberately accept disputed ancestry in exchange for empirically better recovery.

3.5 Complexity and monotonicity

Let B be the bytes read, $n = |T| + |S| + |U| + |C|$ the number of nodes, $m = |E|$ the number of dependencies, and C_{ref} the total reference-evaluation cost. With constant-cost predicate evaluation, streaming decode and worklist traversal require $O(B + n + m + C_{\text{ref}})$ time and $O(n + m)$ index space, excluding reference workspace. Vector predicates add their own elementwise cost. Sorting otherwise unordered updates adds $O(|U| \log |U|)$, as in the prototype. These bounds distinguish payload size from trajectory count.

For fixed G , $M_1 \subseteq M_2$ implies $K_1 \subseteq K_2$. Thus relaxing a tolerance so that fewer events mismatch cannot enlarge the closure. There is no analogous guarantee across arbitrary replacement references: each can expose a different set of failures. This is why a closure must be labeled by the full recipe rather than treated as a permanent property of a checkpoint.

4 Implementation and Its Scope

The evaluated system has two layers. The RTD library provides typed records, an append-oriented store, rescoring, comparison, and lineage queries. A small diagnosis layer, Loom,

loads a recipe and formats a recovery plan. We use the name RTD for the abstraction and do not attribute new training algorithms to the surrounding command-line tool. Existing framework outputs supply most of the payload, avoiding duplication of the trainer or rollout engine.

4.1 Physical store and ingestion

The default backend stores run metadata in `run.json`, trajectories in per-step JSONL shards, and signals, updates, and checkpoints in separate JSONL files. Trajectory payloads contain prompt, response, ground truth, and optional engine fields; signals contain their producer and version. Updates name the trajectory and signal identifiers consumed. Checkpoints name their producing update and physical location. Recomputed readings live in a separate cache, preserving the original measurements.

For the VeRL case, `trainer.rollout_data_dir` emits the input, output, reference answer, and recorded scalar score. An offline ingester assigns identifiers, associates each step’s records with an update, and registers saved checkpoints. It restores verifier-version labels from known phase metadata when native dumps omit those strings. Consequently, the provenance of those labels is the phase configuration, not a per-record version attestation emitted by the evaluator. A deployment with mixed verifier versions within a phase would need finer capture.

This implementation is a *text-verifier profile* of the contract. Native decoded outputs suffice for the reported symbolic rescoring. They do not establish retention of exact action tokens, every engine log-probability, routing decisions, or an exact per-update behavior-model snapshot. The ingester’s policy-checkpoint field can point to the latest saved checkpoint, even though unsaved intervening updates changed the policy. It is therefore a coarse provenance anchor rather than a promise that regenerating from that checkpoint reproduces the response. Our reported localization uses consumed signals and the sequential update order, not exact replay from that anchor.

4.2 Query execution

RESCORE streams trajectories through a Python reference callable. DIFF compares the stored score with the returned value and reports mismatches and per-step counts. A graph index holds identifiers, steps, consumption lists, and checkpoint relationships rather than full token payloads. The sequential closure implementation finds the earliest consuming update and includes the suffix of that weight history. This is a specialization of state-continuation reachability for a single branch, not an implemented arbitrary distributed DAG traversal.

The current comparator maps one selected signal kind to a value per trajectory, using the last encountered value. Thus the experiments rely on one audited reward signal per trajectory. The richer signal-identity contract in Section 3 specifies how multi-verifier or multi-consumption histories should be handled; the prototype is not claimed to implement those cases correctly without adaptation. Likewise, missing signals are skipped by the primitive comparator, so complete counts and referential integrity must be checked before interpreting an empty mismatch set as evidence of agreement.

Caching is keyed by a caller-provided reference identifier. Callers must change that identifier when code or dependencies change, and must not reuse a whole-store cache after extending the store. A robust service would key on a manifest digest, reference identity, signal selection, and all query parameters. We disable the RTD result cache for the reported staged timings. Reference outputs used in the scaling benchmark are explicitly precomputed labels, described separately below.

4.3 Integrity and failure semantics

The JSONL backend is a small single-writer prototype. Immutability is an API convention; it is neither transactional across record files nor cryptographically enforced. Buffered records require flushing, and a process failure may leave an incomplete batch. A deployment

Study	Unit and scope	What it tests
Arithmetic proxy exploitation	One 82,225-parameter GRU run	Historical rescoring, ordinary re-sampling
Arithmetic regression	Three seeded runs; 1,000 updates each	Localization and fresh recovery to a fixed target
Subgroup / data salvage	Separate controlled studies; one seed each	Aggregate masking; limits of naive reuse
VeRL incident	One 130-update, 41,600-trajectory history	Real-framework diagnosis and checkpoint selection
VeRL recovery	Two original 40-update arms; one later supplementary arm	Outcomes under a repaired verifier
Raw capture	Three independent paired 1-GPU process repetitions	Inclusive capture-time difference
Query scaling	Five processes at each of seven store sizes	Dataflow cost, with precomputed reference labels

Table 2: Evidence map. Synthetic store replication expands the query benchmark, not the number of training incidents.

that needs crash-safe forensic coverage should commit an update manifest only after its referenced evidence is durable, validate unique identifiers and expected counts, and make gaps explicit. These are requirements of the contract, not evaluated crash-consistency guarantees of the current writer.

The experiment analyzers verify complete step sets, records per update, and available configuration and identity checks. These checks detect specific capture defects but do not prove an adversarial recorder honest. For the capture benchmark, validation additionally checks 64 prompt groups with five distinct rollout sessions each and consistency of the prompt and ground truth within each group. Such accounting matters: an apparently complete file with duplicate trajectories would not support the completeness assumption used in the closure proposition.

4.4 Recovery interface

A plan returns the reference identifier, first affected update, affected checkpoint set, and eligible restart artifact. Fresh rollouts under the repaired verifier provide the subsequent on-policy training data. The plan does not edit checkpoint weights or erase the influence of disputed measurements from an affected checkpoint. Policy-only restoration and full optimizer restoration are different execution procedures and must be recorded as such.

More generally, data salvage is an optional operation outside this contract. Corrected labels alone do not make historical samples on-policy. Behavior cloning on a selected accepted subset may be useful, but its utility needs separate evaluation. Section 6 includes a toy study showing both successful selective cloning and destructive naive stale-policy updates; neither outcome yields a universal reuse rule.

5 Experimental Design

We organize the evaluation around four questions: whether retained evidence exposes disagreements hidden by aggregate measurements; whether localization changes a recovery decision; how dependent ancestry relates to actual recovery; and what storage, capture, and query costs the implementation incurs. Every result is tied to an experimental unit. Thousands of trajectories within one evolving run do not constitute thousands of independent training replications.

5.1 Controlled arithmetic studies

The prototype uses a GRU policy (Cho et al., 2014) with 82,225 parameters and a 17-token vocabulary. A prompt contains three integers (a, b, c) and target $a + b + c$. A reference checker parses the response as an arithmetic expression, requires exactly the supplied numbers, and verifies the result. The injected proxy instead rewards a response containing the target string. The prompt construction separates canonical expressions from target-copy outputs, so the proxy can reject correct expressions and accept invalid answers. This intentionally simple fault makes the expected disagreements interpretable; it is not a realistic unknown-hack benchmark.

The reward-hacking experiment uses 800 teacher-forcing warm-start steps, achieving approximately 94.3% sampled reference accuracy, followed by 1,000 GRPO updates. Each update samples 16 prompts and eight completions per prompt. The learning rate is 3×10^{-4} and the entropy coefficient 0.02. Checkpoints are saved every 25 updates. A separate re-sampling check compares 100 historical outputs with outputs from the same checkpoint and sampling configuration using a different seed.

The verifier-regression study is designed to preserve meaningful progress before the fault. It starts from a weaker 500-step warm-start (approximately 0.42 sampled accuracy in seed 0), trains with the correct verifier for 500 updates, and then uses the substring proxy for the next 500. The learning rate is 10^{-3} with the same batch/group sizes and entropy coefficient. The study uses three seeds. The toy logs number updates from zero: update 500 is the first faulty update, while checkpoint 500 is saved *before* it. This differs from the VeRL post-update checkpoint convention.

Recovery from the closure boundary, latest checkpoint, a manual checkpoint at step 750, or the warm-start state uses fresh rollouts and the corrected verifier for up to 600 updates. The target is a sampled training-batch reference reward of at least 0.8; reported tokens count through the first qualifying batch. A target reached at recovery index zero still costs that first batch’s tokens. It does not mean convergence was independently established or that no computation was needed. Behavioral bisection probes saved checkpoints to classify their sampled accuracy; its diagnostic tokens are accounted separately.

The partial-regression study modifies scores only for prompts whose target is at least 20. We report overall and slice-level corrected accuracy over the final 50 updates, together with the predicate concentration in the mismatch set. The trigger is known by construction, so perfect concentration is a mechanism check rather than out-of-sample subgroup discovery. A separate sparse-checkpoint salvage experiment starts from checkpoint 300 of the seed-0 regression history and compares fresh training, cloning accepted outputs from different history windows, and naive stale GRPO. Its target is 0.85, distinct from the main regression study’s 0.8 target.

5.2 VeRL/MATH incident and recovery

The retained incident uses GRPO in VeRL (Sheng et al., 2025) with a Qwen2.5-Math-1.5B actor (Yang et al., 2024) and the DigitalLearningGmbH/MATH-lighteval distribution of MATH problems (Hendrycks et al., 2021). It contains 320 trajectories per update, 60 updates under the intended math checker, and 70 under a length-based proxy. The proxy assigns zero if no boxed-answer marker appears and otherwise

$$r_{\text{proxy}}(y) = \max(0.1, 1 - |y|_{\text{char}}/1500).$$

It rewards short formatted responses independently of answer correctness. The reference uses VeRL’s symbolic math grader, with the wrapper’s numeric answer-comparison fallback. Scores and accuracies are relative to this implementation, not human mathematical adjudication.

The audit recipe compares pass/fail at threshold 0.5 on recorded and reference values. Saved states occur every ten updates plus the initial state, giving 14 checkpoint records. In VeRL’s convention, checkpoint 60 is the state after update 60, and the first faulty update is 61. The actor is initialized from the Base model, and the faulty phase and recovery reuse an

Instruct run configuration. The sidecar therefore records the model as Instruct, whereas the retained checkpoint configuration has Base-style EOS metadata. We do not use this incident to claim a Base-versus-Instruct comparison.

The original recovery arms resume checkpoints 60 and 130 with the repaired verifier for 40 additional updates and 12,800 fresh trajectories each. They match update count, not generated token count, wall time, or starting policy. We report final-batch accuracy and the mean of the 40 batch accuracies, not held-out MATH test accuracy. The observed training batches need not contain the same prompts across different starting steps, so differences are descriptive outcomes rather than a paired causal estimate of checkpoint choice.

A later checkpoint-80 arm uses the same 40-update/320-trajectory budget. It is reported separately because it was executed later and its original post-run validator failed on scheduler job-ID formatting before running scientific gates. A later repair of the validator revalidated the stored run. Our independent artifact check verifies score-summary agreement, update coverage, identifiers, and compact-file hashes, but does not repeat the entire preregistered validation or independently regrade model outputs. Appendix C records this provenance boundary.

5.3 Storage and runtime experiments

Physical storage is measured over native rollout dumps, the RTD evidence and lineage sidecar, derived reference cache, and retained checkpoints. All MB/GB quantities are decimal. The sidecar includes its own payload copy, so it is compared with native dumps rather than interpreted as a pure index overhead. A single CPU timing separates fresh ingestion, in-process reference evaluation, comparison, graph construction, closure, and planning. The 2.99-second query-stage sum excludes process startup and store opening; a separate fresh command invocation takes 7.82 seconds. Neither timing estimates deployment-wide capture overhead.

Raw capture cost uses three paired fresh-process repetitions on one NVIDIA A100-SXM4-40GB. Each arm has two warm-up updates and ten measured updates; enabled arms write all 12 updates. Arm order alternates. The semantic configuration difference is native trajectory-dump capture; checkpoint saving, validation, and tracing are disabled. Timing includes the measured foreground window and final asynchronous-writer drain. For pair i , $\delta_i = (t_i^{\text{on}} / t_i^{\text{off}}) - 1$. We summarize the three pair differences with their mean and a descriptive Student- t interval, treating the process pair as the independent unit. This benchmark measures raw capture, not simultaneous sidecar indexing or reference evaluation.

The query benchmark uses one SQUID CPU node with two Intel Xeon Platinum 8368 sockets (76 physical cores available). Seven warm-up processes precede 35 randomized measured processes, five per size. The OS page cache remains warm and the RTD result cache is disabled. Reference values are precomputed outcomes looked up in constant time, so the benchmark isolates dataflow work rather than symbolic grading. Sizes 10,240, 24,960, and 41,600 are real complete-update prefixes; larger sizes are synthetic lineage replications with unique identifiers. The reported time sums rescore, diff, graph construction, and closure and excludes process startup and store opening. Available CPU cores do not imply that the Python query saturates all of them.

6 Results

6.1 Retained evidence reveals hidden failures

In the controlled reward-hacking run, sampled reference accuracy falls from 0.89 to below 0.05 within 50 updates while the flawed training reward subsequently rises. Rescoring the 128,000 retained trajectories preserves the actual outputs on which this divergence occurred. The separate re-sampling check reproduces only 2 of 100 outputs with a different seed. This is evidence that ordinary re-sampling is not a substitute for the retained history, not a test that exact checkpoint-plus-random-state replay is impossible. The toy store uses 85.84

Quantity	Seed 0	Seed 1	Seed 2
Bisection diagnostic tokens	13,718	15,376	14,185
RTD diagnostic rollout tokens	0	0	0
Bisection selected checkpoint	525	500	500
RTD selected checkpoint	500	500	500
RTD recovery tokens to 0.8	768	22,291	768
Warm-start restart tokens to 0.8	207,394	229,049	179,745
Manual checkpoint 750 tokens to 0.8	127,135	not reached	not reached
Latest checkpoint 1000 tokens to 0.8	384,760	not reached	not reached

Table 3: Controlled regression: diagnosis above the divider, fresh recovery below it. “Not reached” means within 600 recovery updates. Recovery from bisection’s selected checkpoint is not an additional measured arm. The target is the first sampled training batch at or above 0.8.

bytes per token in the particular 50-step capture trial; this encoding-specific number is not extrapolated to the LLM case.

The partial regression is more revealing than the aggregate collapse. Over the final 50 updates, aggregate training reward is 0.81 and overall corrected accuracy is 0.87, essentially its pre-fault level. However, corrected accuracy is only 0.43 on the affected 13.7% of prompts versus 0.93 elsewhere. All 5,483 retained mismatches satisfy the injected predicate. Its store base rate is 13.84%, giving $7.22\times$ enrichment in the mismatch set. A CPU rescore takes 2.78 seconds without new rollout.

These results support two different conclusions. First, rescoring can separate a recorded proxy from a chosen reference. Second, complete per-trajectory evidence preserves the information needed to inspect input slices after the failure, even when an aggregate does not expose their deterioration. They do not show an algorithm discovering an unanticipated predicate or defeating an adaptive adversary. The observed perfect concentration partly follows from the injected fault design.

6.2 Localization and recovery costs are different quantities

The three-seed regression study locates faulty update 500 and selects pre-update checkpoint 500 in every seed. This exact onset is a sanity check of capture, disagreement, and consumption logic. It would be misleading to interpret agreement with a constructed fault boundary as a statistical detection result. The more operational question is what checkpoint is selected and what subsequent recovery requires.

Table 3 keeps the accounting separate. Bisection selects already affected checkpoint 525 in seed 0 because behavioral degradation lags consumption of faulty measurements. RTD needs no new diagnostic rollouts, but still incurs CPU parsing and reference-evaluation cost. Recovery from its selected checkpoint reaches the target with 768/22,291/768 tokens, compared with 207,394/229,049/179,745 from the warm-start state. In seeds 0 and 2, the selected checkpoint already produces a qualifying first recovery batch; the 768 tokens measure that batch rather than learning from a collapsed policy in one step.

This is not an end-to-end latency comparison with bisection. We did not measure recovery from each bisection-selected checkpoint as a separate arm, and diagnostic token counts exclude costs of evaluating the reference. The result instead shows that lineage can preserve a longer eligible prefix than restart while avoiding the behavioral probe’s incorrect dependency classification in one seed.

6.3 A real-framework incident and its dependency boundary

The VerL incident retains 41,600 trajectories over 130 updates. Thresholded disagreement identifies 15,560 mismatches (37.40%), first consumed at update 61. Sequential closure marks 70 updates and seven saved checkpoints and selects checkpoint 60. At the faulty-phase

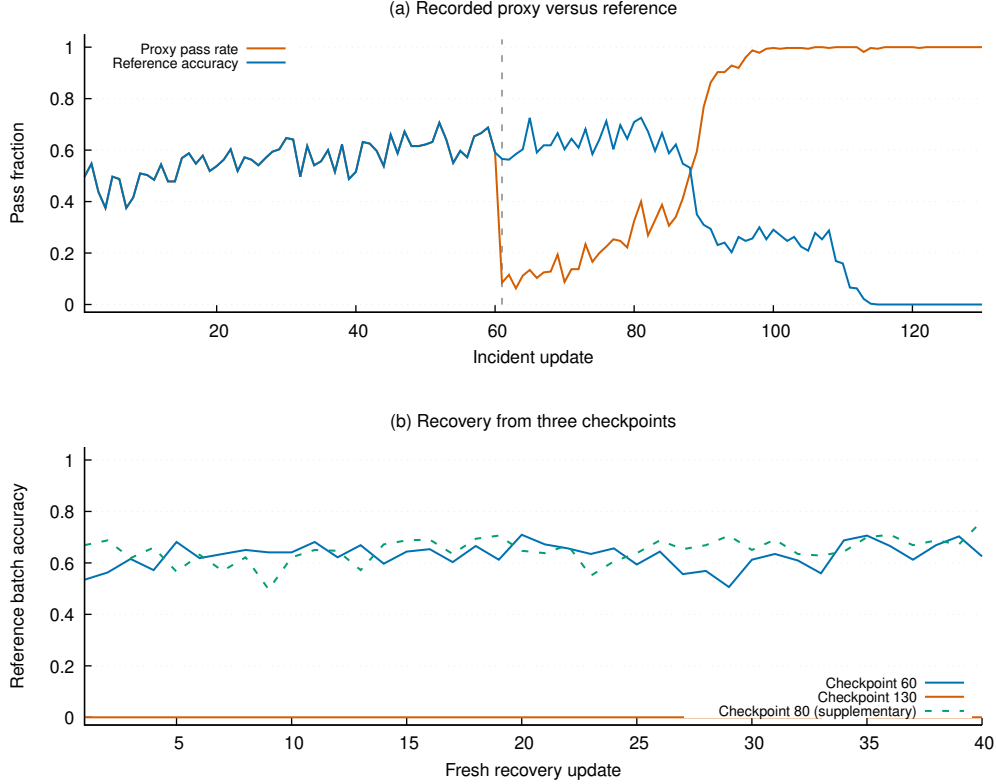


Figure 2: The retained VeRL incident and fresh recovery outcomes. Top: the dashed line marks faulty update 61; proxy pass rate uses threshold 0.5. Bottom: each point is a 320-trajectory training-batch reference accuracy, not a held-out score. Checkpoint 80 is a later supplementary arm with the validation limitation described in Appendix C. The curves are single trajectories of training, not across-seed means.

endpoints, mean proxy reward rises from 0.1831 to 0.9947 while reference accuracy falls from 56.56% to zero. Mean response length falls from 1,742.73 to 8.00 characters. Figure 2 plots proxy *pass rate*, which differs from mean continuous reward.

Simple checkpoint-selection rules disagree with the dependency criterion (Table 4). The best observed reference accuracy among saved-step batches occurs at checkpoint 80 (0.7094), exceeding 0.5906 at checkpoint 60, despite checkpoint 80’s affected ancestry. The first observed accuracy decline selects checkpoint 30, before the incident; the proxy-best rule selects checkpoint 110, breaking its tie with 120 and 130 by choosing the earliest maximum. These statistics are observed training-batch measurements, not a common evaluation set applied to all checkpoints.

6.4 Affected ancestry does not determine recoverability

With the repaired verifier and 40 fresh updates, the original checkpoint-60 arm reaches 62.5% final-batch reference accuracy, while the checkpoint-130 arm reaches zero. Their mean accuracies over recovery are 62.95% and 0.039%, respectively. This is evidence that restoring an earlier reference-unaffected state can be useful compared with resuming this severely degraded latest state. It does not establish that the boundary is the best available starting point.

The later checkpoint-80 arm makes that qualification concrete. Its reported final-batch accuracy is 76.25% and mean recovery accuracy is 65.01%. We verified that all 40 raw batch score

Selection rule	Checkpoint	Affected updates retained	Unaffected updates lost
Latest saved	130	70	0
Best observed reference batch	80	20	0
Before first reference decline	30	0	30
Best proxy pass rate (earliest tie)	110	50	0
Latest outside RTD closure	60	0	0
Onset-aware oracle	60	0	0

Table 4: Retrospective selection under different objectives. Counts are relative to the observed onset at update 61. Matching the onset-aware oracle is a structural result, not proof of best recovered accuracy.

Starting checkpoint	Final batch	Mean of 40 batches	Evidence status
60: outside closure	62.50%	62.95%	Original recovery arm
130: inside closure	0.00%	0.039%	Original recovery arm
80: inside closure	76.25%	65.01%	Later supplementary arm; partial independent audit

Table 5: Recovery with 40 updates and 12,800 fresh trajectories per arm. Budgets match in updates and trajectories, not necessarily tokens or wall time. All accuracies are training-batch reference measurements.

means agree with the stored reference summaries, with 12,800 distinct recorded trajectory identifiers and complete update coverage. Its validation provenance is less complete than a fully independently gated replication, and the training was not contemporaneous with the other arms. We therefore report it as supplementary, without a significance test or a matched causal superiority claim. Nevertheless, omitting it would overstate the available evidence for always discarding affected checkpoints.

A useful recovery interface should therefore expose both ancestry and behavioral evidence. Operators requiring exclusion of disputed training signals can select checkpoint 60. Operators optimizing recovery utility may evaluate checkpoint 80 despite its ancestry. RTD supplies the constraint and the history; it does not collapse these different objectives into a single notion of “cleanest is best.”

6.5 Retaining data does not justify arbitrary reuse

In the toy sparse-checkpoint study, recovery begins from checkpoint 300 at reference accuracy 0.7686. Two hundred naive stale-GRPO updates using stored batches with corrected rewards reduce accuracy to 0.0010. This negative result supports caution about treating retained evidence as on-policy samples. It does not prove that all offline or off-policy methods must fail.

Cloning reference-accepted outputs from the unaffected window [300, 500) instead produces 0.9248 post-cloning accuracy using no new rollout tokens. The subsequent first qualifying fresh batch costs 768 tokens, compared with 37,650 for fresh-only recovery to target 0.85. Cloning accepted outputs from the affected window produces 0.7432 post-cloning accuracy and uses 41,454 subsequent tokens to target. These are single-seed outcomes, not a guarantee that a closure partition certifies data quality. They show why an audit substrate and a reuse algorithm should be specified and evaluated separately.

6.6 Physical storage and diagnostic latency

The real-incident sidecar occupies 91.491636 MB excluding its 2.128705 MB reference cache. Native VeRL dumps occupy 70.338237 MB. The sidecar is 30.07% larger as a standalone representation; retaining both costs their sum, not their difference. Independently gzipping the files yields 14.654025 MB for RTD and 12.543512 MB for native dumps. Checkpoints

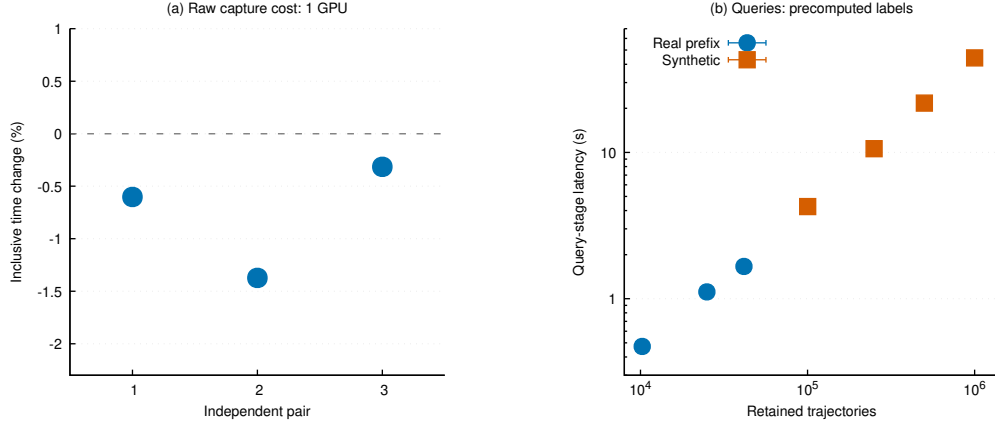


Figure 3: Separate systems measurements. Left: three paired single-GPU raw-capture time differences; no claim of acceleration follows from negative estimates. Right: warm-filesystem query-stage medians and IQRs, using precomputed reference labels. Orange points are synthetic lineage replications, not additional training incidents.

occupy 253.164077 GB, but comparison with checkpoint size is physical accounting, not evidence that storing trajectories is free or that checkpoints and evidence are interchangeable.

Tokenizing the retained prompt and response strings gives 20.833430 million text tokens and a footprint of 4.392 bytes per text token (5.640 per response token). These denominators are reconstructed from text, not a verified log of exact engine action-token counts, and inherit the Base/Instruct labeling difference in the historical model and tokenizer metadata. They should not be compared directly with the toy 85.84-byte value as an implementation speedup or compression result.

Fresh offline ingestion takes 14.51 seconds. The real-reference query stages take 1.675 seconds for rescore, 0.205 for diff, 1.090 for graph construction, 0.020 for closure, and less than a millisecond for planning, for a total of 2.989 seconds. These are single observations. A fresh CLI invocation takes 7.821 seconds including additional overhead, while a cached invocation takes 1.393 seconds. The headline 2.99-second number therefore describes a query-stage measurement, not total incident response time or a cold-start service-level objective.

6.7 Capture cost and query scaling

The three paired single-GPU capture differences are -0.602% , -1.373% , and -0.316% . Their mean is -0.764% , with a descriptive 95% t interval of $[-2.122\%, +0.595\%]$. We detect no slowdown in this small workload sample; negative differences are not evidence that capture accelerates training. Each enabled process writes 3,840 records occupying 9.35–9.42 MB, and all integrity gates pass over 11,520 records. A separate two-GPU pilot reports $+4.284\%$ for one pair. It is included as a limitation on extrapolation, not as a scaling trend: the pilot has one pair, different execution provenance, and a disclosed teardown anomaly (Appendix D).

The dataflow benchmark grows from 0.472 seconds at 10,240 trajectories to 44.209 seconds at one million (Table 6). At the largest size, graph construction accounts for a median 24.069 seconds, rescore label lookup 14.472, diff 4.790, and closure 0.792; component medians need not sum exactly to the median of totals. Memory grows to approximately 1.68 GiB. The result supports tractable query costs at this scale but exposes graph construction and indexing as substantial costs. The 1.663-second point at 41,600 must not replace the 2.989-second real reference result: the reference work and measurement boundary differ.

Trajectories	Input	Median (s)	IQR (s)	Max RSS (MiB)
10,240	Real prefix	0.472	0.472–0.473	31.1
24,960	Real prefix	1.113	1.112–1.115	57.3
41,600	Real prefix	1.663	1.653–1.675	81.7
100,160	Synthetic	4.263	4.229–4.276	186.9
250,240	Synthetic	10.595	10.591–10.618	434.0
500,160	Synthetic	21.731	21.700–21.782	870.8
1,000,000	Synthetic	44.209	43.889–44.211	1,718.7

Table 6: Query-stage scaling: five independent processes per size, warm filesystem, result cache disabled. RSS is the median process maximum. Constant-time reference-label lookup replaces the symbolic grader here.

7 Related Work

Reward hacking and measurement validity. Measurement validity concerns the relationship between observable scores and the intended construct (Jacobs & Wallach, 2021), and optimizing an imperfect proxy can worsen the intended outcome (Skalse et al., 2022). Production reward-hacking studies motivate the gap between a reward signal and the intended task (MacDiarmid et al., 2025). Monitoring reasoning can reveal misconduct, but optimizing against the monitor can incentivize concealment (Baker et al., 2025). RTD is complementary: it retains historical observations for a future audit, rather than providing a new detector of deception or a verifier resistant to gaming. Its reference-relative semantics avoid conflating disagreement with knowledge of the true objective.

Lineage and hindsight logging. RDDs recover data by replaying logged deterministic transformations (Zaharia et al., 2012). Flor enables hindsight logging by restoring and re-executing training with additional instrumentation (Garcia et al., 2021). RTD adopts the same reasoning about reconstructability but makes the supported historical measurement query explicit. It retains outputs and consumed readings when the available replay contract cannot recover them. It does not claim that prior replay systems cannot handle randomness or that every RLVR rollout is fundamentally impossible to replay. The distinct design choice is the combination of immutable measurement evidence, actual consumption lineage, and reference-conditioned closure queries.

LLM reinforcement-learning systems. HybridFlow organizes efficient distributed RLHF dataflows (Sheng et al., 2025); AReaL overlaps generation and training and accounts for stale behavior policies (Fu et al., 2025). Such frameworks already carry many relevant tensors and identifiers. The RTD proposal is not to replace those systems or invent output logging, but to turn their emitted evidence into a durable audit contract. Table 7 compares the cited designs, without asserting that no later release can provide a particular field.

Training correctness and mismatch repair. TrainVerify checks equivalence of distributed training plans (Lu et al., 2025), and Mycroft traces collective communication to localize reliability problems (Deng et al., 2025). These address correctness or infrastructure dependencies at different layers from historical reward semantics. Routing replay and training–inference mismatch analysis (Ma et al., 2025; Zhong et al., 2026) identify additional observations an RTD profile may need to retain. They also show why materialization should be driven by the intended question: storing text is sufficient for some verifier audits but insufficient for replaying expert-routing decisions. RTD provides an evidence interface, not an alternative to the algorithmic corrections these systems implement.

Design	Existing objects / objective	RTD’s additional audit contract
HybridFlow	Rollout/training tensors, distributed data dependencies, efficient execution	Durable consumed outputs and verdict identities; per-record links to update and saved-state ancestry
AReal	Asynchronous samples, behavior probabilities, policy versions, staleness-aware optimization	Preserve relevant events beyond scheduling; resolve the actual signal versions consumed by each update
Flor	Checkpoints and execution state for replay and hindsight logs	Retain non-replayable measurement inputs; compare replacement readings without regenerating historical events
RTD	Query-relative base evidence and typed lineage	Explicit reference/predicate/coverage identity; dependency closure and eligible rollback prefix

Table 7: Comparison of organizing interfaces. Existing fields can satisfy RTD requirements without duplicate capture. General data lineage and logging precede RTD; the contribution is their specified application to historical RLVR measurement and recovery constraints.

8 Discussion and Limitations

8.1 The reference is an input, not a discovery mechanism

The experiments deliberately retain a checker that can identify the injected faults. This does not model how operators discover or repair an unknown verifier in a real incident. RTD becomes useful once a new hypothesis can be expressed as an evaluator or disagreement predicate. A reference can arrive later through a repaired parser, new unit tests, human adjudication, or a stronger model; its provenance and limitations must remain visible. If both evaluators share a blind spot, disagreement queries will not flag it. An empty result should be read as “no observed disagreement under this recipe and capture scope,” not “the run is correct.”

Stochastic references pose an additional issue. Repeated judge calls may return different readings on the same output. An auditor must either retain those readings and the sampling protocol, or define a statistical query over repeated judgments. Our deterministic closure theorem applies to the resulting fixed mismatch set, not to the statistical validity of that set. Human or learned references can also be expensive; zero new rollout tokens says nothing about their evaluation cost.

8.2 Coverage versus sampling

A sampled audit can detect a common failure without retaining every trajectory. Under independent sampling of events with mismatch probability p , the probability of missing all mismatches in m samples is $(1 - p)^m$. This can guide aggregate monitoring, but RL trajectories are dependent and the relevant rate may vary by input slice and time. More fundamentally, a sample that finds a failure does not identify every update that consumed it. Exact recorded-history closure requires complete coverage of the dependencies claimed by the query.

Complete capture does not require storing every intermediate tensor. It requires retaining the sufficient evidence for the declared profile. Compact identifiers, deduplicated prompts, and recomputation of derived quantities can reduce storage. A partial-retention system should expose which questions remain answerable rather than present sampled evidence as a complete forensic record. The subgroup result motivates that coverage distinction without implying that sampled monitoring is useless.

8.3 Recovery has multiple objectives

Excluding disputed dependencies, minimizing fresh rollout cost, and maximizing final accuracy are different objectives. The latest eligible prefix is attractive when provenance exclusion is required, and our original recovery comparison shows a large benefit over the latest collapsed state in one incident. The supplementary checkpoint-80 result shows why that benefit must not become a general optimality claim. Checkpoint 80 retains learning after the first mismatches and remains recoverable. Closure deliberately ignores whether that learning was numerically harmful, beneficial, or later overwritten.

A practical planner could present several options: a dependency-excluding rollback, a behaviorally promising affected checkpoint, or a fresh restart, together with their evidence and costs. Choosing among them requires utility measurements, domain constraints, and possibly new recovery trials. RTD can make the alternatives auditable; it cannot select a universal winner from ancestry alone. Similarly, useful behavior-cloning salvage in the toy experiment does not license arbitrary policy-gradient reuse of stale samples.

8.4 External validity and implementation limits

The arithmetic workload is deliberately small and its failures trivial. The real-framework evidence is one injected regression on a 1.5B-parameter model, not an evaluation of naturally occurring reward hacks, long-horizon tools, frontier models, or multiple independent fault families. Recovery accuracies are training-batch statistics from single arms; no held-out generalization or across-seed confidence interval is claimed for them. The capture-cost study has three independent pairs, and the two-GPU pilot has one. The query-scaling curve expands stored lineage synthetically, not training scale.

The historical artifact collection also has provenance limitations: model-variant labels differ between the sidecar and checkpoints, the later recovery required a validator repair, and the full general graph contract exceeds the sequential implementation. The reference wrapper’s fallback behavior must be included in its identity, since silently changing dependencies could change grading semantics. The JSONL recorder has no evaluated transactional durability or tamper resistance. These are concrete limits of the current evidence, not implementation details that can be assumed away by the formal specification.

8.5 Data stewardship

Materialized prompts, responses, and tool outputs may contain private or licensed information. A production audit store inherits their retention and access requirements. Redaction can improve privacy but may remove inputs required by future references; deleting evidence should therefore also change the reported audit coverage. The present experiments concern arithmetic and MATH data, and do not evaluate privacy-preserving capture. The compact paper artifacts retain numerical summaries and provenance rather than republishing raw model-output corpora.

Artifacts. The RTD library, a runnable example, the compact numerical evidence, its validation script, and the figure-generation commands are available at <https://github.com/PeterWrighten/rtd>; the numerical package is under experiments/paper_evidence. The package distinguishes locally audited raw artifacts from portable summary-level reproduction.

9 Conclusion

RTD treats historical reward measurements as auditable data with explicit consumers. Its retention rule is query-relative: preserve what cannot be reconstructed under the available replay contract, and derive other quantities only with pinned dependencies. Rescore, diff, and localize identify disagreements and their recorded dependency closure, providing a precise rollback constraint without claiming access to an oracle evaluator or the best recovery policy.

The experiments show why this distinction is useful. Complete evidence reveals a degraded subgroup masked by aggregate accuracy; a real VeRL incident supports inexpensive retrospective diagnosis; and recovery outcomes demonstrate both the value and the limits of excluding disputed ancestry. Raw capture and query costs are measurable and workload-specific. The resulting contribution is a concrete infrastructure contract for post-training audits, with explicit evidence boundaries on what it can preserve, diagnose, and justify.

References

- Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. arXiv preprint arXiv:2503.11926, 2025. URL <https://arxiv.org/abs/2503.11926>.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734, 2014. doi: 10.3115/v1/D14-1179. URL <https://aclanthology.org/D14-1179/>.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Yangtao Deng, Lei Zhang, Qinlong Wang, Xiaoyun Zhi, Xinlei Zhang, Zhuo Jiang, Haohan Xu, Lei Wang, Zuquan Song, Gaohong Liu, Yang Bai, Shuguang Wang, Wencong Xiao, Jianxi Ye, Minlan Yu, and Hong Xu. Mycroft: Tracing dependencies in collective communication towards reliable LLM training. arXiv preprint arXiv:2509.03018, 2025. URL <https://arxiv.org/abs/2509.03018>.
- Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. arXiv preprint arXiv:2505.24298, 2025. URL <https://arxiv.org/abs/2505.24298>.
- Rolando Garcia, Eric Liu, Vikram Sreekanti, Bobby Yan, Anusha Dandamudi, Joseph E. Gonzalez, Joseph M. Hellerstein, and Koushik Sen. Hindsight logging for model training. *Proceedings of the VLDB Endowment*, 14(4):682–693, 2021. doi: 10.14778/3436905.3436925. URL <https://www.vldb.org/pvldb/vol14/p682-garcia.pdf>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Abigail Z. Jacobs and Hanna Wallach. Measurement and fairness. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021. doi: 10.1145/3442188.3445901. URL <https://arxiv.org/abs/1912.05511>.
- Yunchi Lu, Youshan Miao, Cheng Tan, Peng Huang, Yi Zhu, Xian Zhang, and Fan Yang. TrainVerify: Equivalence-based verification for distributed LLM training. In *Proceedings of the Symposium on Operating Systems Principles (SOSP)*, 2025. URL <https://arxiv.org/abs/2506.15961>.
- Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing MoE reinforcement learning by aligning training and inference routers. arXiv preprint arXiv:2510.11370, 2025. URL <https://arxiv.org/abs/2510.11370>.
- Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex

- Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural emergent misalignment from reward hacking in production RL. arXiv preprint arXiv:2511.18397, 2025. URL <https://arxiv.org/abs/2511.18397>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. doi: 10.1145/3689031.3696075. URL <https://arxiv.org/abs/2409.19256>.
- Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/3d719fee332caa23d5038b8a90e81796-Abstract-Conference.html.
- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. arXiv preprint arXiv:2409.12122, 2024. URL <https://arxiv.org/abs/2409.12122>.
- Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 15–28, 2012. URL <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>.
- Tianle Zhong, Neiwen Ling, Yifan Pi, Zijun Wei, Tianshu Yu, Geoffrey Fox, Peng Wu, and Xiao Yu. Diagnosing training inference mismatch in LLM reinforcement learning via a zero-mismatch reference. arXiv preprint arXiv:2605.14220, 2026. URL <https://arxiv.org/abs/2605.14220>.

A Worked Lineage Examples

Consider three updates u_1, u_2, u_3 with logical states c_0, c_1, c_2, c_3 . Suppose only c_0 and c_3 are physically saved, and u_2 consumes a signal that disagrees with the reference. The closure contains u_2, c_2, u_3, c_3 even if every signal in the third batch agrees: u_3 starts from c_2 . The eligible saved rollback is c_0 , so excluding disputed dependencies also discards unaffected update u_1 . If c_1 had been saved, it would instead be selected. This illustrates why checkpoint spacing and capture granularity are separate decisions.

Now let trajectory t have two signals s_a and s_b , but suppose u_2 consumes only s_b . A reference disagreement with s_a alone does not place u_2 in the closure. Starting traversal from t would incorrectly include both readings’ consumers. The sequential experimental profile avoids this ambiguity by storing one audited reward signal per trajectory; the general contract preserves consumed signal identifiers explicitly.

Finally, suppose two branches resume from c_1 . A disagreement consumed only on branch A reaches its descendants, not branch B, unless a later operation imports affected state or data from A into B. A chronological “all updates after onset” suffix is invalid for this case. The full DAG specification uses actual state and consumption edges. Our evaluated prototype’s suffix implementation applies only to a single sequential weight history.

B Additional Reproducibility Details

B.1 Metric definitions

For a retained batch B_k of size n_k , reference accuracy is $n_k^{-1} \sum_{t \in B_k} \mathbf{1}[r^*(t) \geq 0.5]$ and proxy pass rate uses the same threshold on recorded signals. Mean continuous reward is $n_k^{-1} \sum r(t)$ and can differ from the pass rate. All real-incident batches have 320 trajectories. Recovery mean accuracy is the arithmetic mean of the 40 batch accuracies; because batch sizes match, it also equals the reference pass fraction over all 12,800 trajectories. Final-batch accuracy is the last batch’s value. Neither is a held-out test score.

The reference-conditioned affected checkpoint count includes only saved states produced by affected updates, not every logical state. A direct mismatch count, an affected update count, and an affected checkpoint count are different denominators. In the real incident they are 15,560, 70, and seven, respectively. Text-token storage ratios retokenize saved strings and should not be interpreted as exact action-token capture statistics.

B.2 Source-to-result mapping

The accompanying numerical package contains per-update incident and recovery series, the toy diagnostic/recovery summaries, capture pair measurements, and the query-scaling trials and summaries. It also includes a collector that reads the existing artifacts without training or writing into their directories, an arithmetic validation script, and figure-generation commands. The figures are regenerated from these compact files. No new model training is claimed by this expanded paper.

The incident summaries originate in `loom.incident.base/loom.diagnosis.json`, with recovery readings in each arm’s `loom.verify.json`. Physical byte counts and single-observation timing values originate in the retained `capture.cost.json`. The reported toy numbers originate in the V4, V5, V6, and V8 JSON summaries. SHA-256 manifests bind the exported summaries to the local inputs; they establish artifact identity, not correctness of the generating experiment.

The RTD and Loom source checkouts inspected for this version are `3a601aa833de` and `d1eb875a1b70`, respectively. These identify the implementation inspected for the scope discussion, not a claim that every historical experiment ran those exact revisions. The capture and scaling studies have their own frozen execution provenance below. A full rerun still requires the training stack, datasets, checkpoints, and raw artifacts; the compact package is sufficient to recompute displayed numerical summaries and figures, not to reproduce all training outcomes independently.

B.3 Exploratory and negative results

The early toy recovery experiment began with a faulty verifier at the start of RL. Its selected boundary coincided with the initial checkpoint, so it could not demonstrate savings over a restart. This motivated the mid-run regression study, in which 500 correct updates create progress worth preserving. We do not pool these designs as independent evidence for one recovery effect.

Likewise, an over-accepting toy verifier that rewards both correct answers and target copies did not induce hacking in the reported exploratory configurations. Equal within-group rewards can remove the advantage incentive needed to move toward the proxy. The substring fault used in the main experiment rejects canonical correct expressions, creating a more direct optimization pressure. The real length-decay proxy similarly creates a preference for shorter formatted outputs. These choices help exercise the diagnosis mechanism but narrow the interpretation of the fault experiments.

C Later Checkpoint-80 Recovery: Provenance and Audit

The later recovery was executed as PBS job 1200011.sqd on August 29, 2026. It completed updates 81–120, with 320 trajectories per update, and wrote checkpoints 90, 100, 110, and 120. The initial post-run validator rejected a comparison between compute-side job ID 0:1200011.sqd and frontend ID 1200011.sqd. This was its first gate, so subsequent scientific checks did not execute in that invocation. The failed marker remains alongside the later completed marker.

A later validator repair, recorded as PostTrain 8bf826a, normalized the scheduler identity and revalidated the stored artifacts. The retained note reports successful manifest, configuration, rollout, checkpoint, store, reference-summary, and ledger gates without rerunning training. It also explicitly states that the preregistered independent validation had not been completed. We preserve that limitation rather than treating the completed marker alone as proof of independent acceptance.

For this paper we performed a separate read-only artifact audit. Across all 40 batches, raw score means exactly match the reference-summary accuracies; the store reports zero mismatches; all expected rollout files contain 320 rows; and the 12,800 raw rows have globally unique nonempty identifiers with matching step labels. We checked 138 compact-file ledger entries against their SHA-256 hashes. Sixteen large or tensor entries were not rehashed. We did not rerun the full validator, re-establish the scheduler/configuration gates independently, or run the symbolic grader again on each output. The outcome is therefore supplementary and less strongly validated than a new fully gated replication.

The final value 0.7625 and mean 0.650078125 are consistent across the retained summaries and raw scores. Their role in the paper is to qualify an overly strong claim that every affected checkpoint is a poor recovery starting point. They are not used to estimate a population effect or to claim that checkpoint 80 is universally preferable to checkpoint 60.

D Systems Measurement Provenance

D.1 Single-GPU capture pairs

The frozen workload used PostTrain bb55364 and VeRL 235f1f3a; the final analyzer is PostTrain dae2d8c. Post-run analyzer changes checked a derived tracing-name alias and parsed JSONL by literal LF boundaries so a legal Unicode line-separator inside an output was not mistaken for an extra record. These were analyzer changes, not new training runs. The pinned environment records Python 3.12.12, PyTorch 2.8.0+cu128, and vLLM 0.11.0. The GPU was an A100-SXM4-40GB on SQUID.

Pair	Capture on (s)	Capture off (s)	Change
1	1604.947820	1614.669927	−0.602%
2	1601.683820	1623.985674	−1.373%
3	1633.572049	1638.751392	−0.316%

Table 8: Foreground measured window plus final writer drain. The small sample supports a descriptive interval, not a high-powered equivalence test.

Default same-seed fresh processes did not reproduce identical stable evidence multisets in this package. The experiment does not isolate data ordering, sampling, and training-state divergence, and therefore cannot show that seed-based replay is impossible under a controlled execution contract. The native writer measurement also does not include running RTD queries concurrently with training.

D.2 Two-GPU pilot

A separate one-pair pilot used workload commit 548aa2382138 and a post-hoc analyzer at 5491433, with VeRL 235f1f3a. Its inclusive capture change is +4.284%. The post-hoc ana-

lyzer corrected checks for derived configuration aliases; the raw workload was unchanged. A data-loader destructor error appears after the final update and completion messages in the disabled arm. The archived assessment classifies it as teardown-only, with exit status zero and no pending writer futures. We disclose it because this is a small pilot, not a clean multi-GPU scaling series. No four- or eight-GPU capture result is reported.

D.3 Query scaling

The query benchmark ran as PBS job 1045762.sqd with PostTrain f19c60335c56. Each store contains complete 320-trajectory updates. Replicated stores receive distinct trajectory, signal, update, and checkpoint identifiers; these are expanded lineage workloads, not new rollouts. The earlier partial-update pilot is excluded. Five measured processes per size and inclusive quartiles produce Table 6. Startup and store-open costs are outside the reported stage sum, and precomputed corrected labels replace symbolic reference work. The separate real-reference timing is retained precisely because the scaling benchmark cannot establish the cost of a more expensive reference.